

SettleIQ

Actual Product Development Plan & Revised Roadmap

This document defines the revised direction after completion of the working MVP foundation. The goal is no longer to mechanically continue the original MVP roadmap, but to develop a stronger, coherent Buildathon-level product while preserving the working core.

1. Current Project Status

Phases 1–11 have established a functioning foundation: ingestion, deterministic reconciliation, exception detection, evidence building, AI investigation, deterministic guardrails, human review, auditability, frontend integration, and end-to-end verification.

- Core backend workflow is built and tested.
- The deterministic engine achieved 100% agreement on the 200-record benchmark dataset.
- The frontend MVP is built and connected to backend APIs.
- AI remains advisory and bounded by deterministic guardrails.
- Human review and audit traceability are implemented.
- The main remaining gap is product experience, workflow clarity, metric realism, and broader dataset usability.

2. Why the Strategy Changes Now

The MVP-first strategy was originally a time-management decision: first guarantee a working system under a limited schedule. AI-assisted development completed that foundation much faster than expected. Therefore, the project should not remain artificially constrained to the original MVP scope.

New objective: build a technically sound, visually clear, demonstrably useful Buildathon-level product—not necessarily production-grade, but substantially more complete and coherent.

3. Product North Star

SettleIQ should feel like an intelligent financial reconciliation platform, not a collection of disconnected technical screens.

Core journey:

Upload Data → Validate → Reconcile → Understand Results → Inspect Exceptions → AI Investigation → Safe System Decision or Human Review → Complete Audit Trail

A user should immediately understand what happened, why an exception occurred, what evidence supports it, what the AI found, what the system decided, and what still requires human attention.

4. Non-Negotiable Architecture Principles

- **Deterministic reconciliation remains the source of truth.** AI does not replace classification logic.
- **AI is advisory.** It explains, investigates, estimates confidence, and recommends an action.
- **Guardrails make autonomous system decisions.** High confidence alone never bypasses deterministic safety rules.

- **Bounded autonomy.** Only explicitly safe cases may auto-resolve; uncertain cases go to human review.
- **Human decisions are explicitly attributed.**
- **Important actions are auditable.**
- **Resolved cases remain inspectable.** Read access is never blocked merely because a case is resolved.
- **Preserve working architecture.** Existing backend logic should change only for a genuine defect or required product improvement.

5. Correct Exception Lifecycle

Exception Detected → Evidence Built → AI Investigation → Guardrail Evaluation → Decision

Outcome	Meaning
AUTO_RESOLVED	Only when recommendation, confidence, evidence grounding, known safe mathematical rule, and
HUMAN_REVIEW	When any guardrail fails, evidence is insufficient, confidence is low, or the case is not safely elig

Important correction: A qualifying high-confidence case should not unnecessarily enter the human review queue. However, 99% confidence alone is not sufficient; deterministic guardrails must also pass.

AUTO_RESOLVED cases remain visible with status, confidence, root cause, evidence, decision reason, and audit history. They are inspectable but do not require another human action.

6. Product Problems to Audit and Correct

- Remove unnecessary workflow complexity and duplicated decision steps.
- Clearly separate automatically resolved cases from cases requiring human action.
- Make confidence meaningful rather than repeatedly showing a decorative value such as 95%.
- Ensure dashboard percentages, summaries, and rates are derived from real system state.
- Simplify navigation and ensure every screen has one clear purpose.
- Place AI, evidence, guardrails, review, and audit information where users naturally expect them.

7. Dataset Generalization Goal

The benchmark dataset remains a regression and accuracy asset, but the product should not feel limited to one known CSV example.

- Support valid payments, settlements, and fees datasets using the documented schema.
- Validate uploaded files dynamically.
- Create reconciliation runs from uploaded data.
- Apply the same deterministic rules to each run.
- Retain benchmark data for testing and demonstrations.
- Clearly distinguish benchmark/testing datasets from user-uploaded operational datasets.

8. Revised Development Roadmap

Stage A — Product Audit & Alignment

Review each screen against the original product vision. Classify elements as Keep, Modify, Remove/Simplify, or Add. Do not make uncontrolled changes yet.

Stage B — Core Workflow Correction

Fix exception lifecycle, auto-resolution routing, human review boundaries, confidence presentation, and metric correctness.

Stage C — UI/UX Refinement

Improve layout, navigation, hierarchy, terminology, visual consistency, and loading/empty/error states.

Stage D — Dataset Generalization

Ensure multiple valid uploaded datasets flow cleanly through the same reconciliation pipeline.

Stage E — AI Experience Refinement

Improve evidence-backed explanations, confidence behavior, recommendation clarity, and fallback presentation without giving AI decision authority.

Stage F — Settlement Q&A;

Add Q&A; after the core workflow is clear. It should answer useful questions about runs, exceptions, evidence, and reconciliation results.

Stage G — Evaluation & Accuracy Reporting

Add richer benchmark scoring and reporting where useful for judging and demonstration.

Stage H — Final Buildathon Polish

Complete regression testing, demo preparation, README, architecture explanation, deployment preparation, and final presentation polish.

9. Recommended Immediate Working Method

- Inspect one screen or workflow at a time.
- Capture what currently exists and what a normal user understands.
- Compare it with the intended product behavior.
- Define the exact correction before implementation.
- Use small, scoped implementation prompts.
- Preserve unrelated files and working backend behavior.
- Build and test after each meaningful change.
- Move forward only after the current screen/workflow is aligned.

10. Screen-by-Screen Audit Order

- 1. Dashboard
- 2. Upload Data
- 3. Reconciliation Results
- 4. Exceptions List
- 5. Exception Detail
- 6. AI Investigation and Confidence Presentation
- 7. Guardrail Decision / Auto-Resolution Presentation

- 8. Human Review Queue
- 9. Audit Logs
- 10. Overall navigation and complete end-to-end journey

11. Definition of Success

- A user can understand the product without technical guidance.
- The path from uploaded data to reconciliation outcome is obvious.
- Metrics are real, dynamic, and mathematically correct.
- Confidence is meaningful rather than decorative.
- Safe qualifying cases auto-resolve without unnecessary human review.
- Unsafe or uncertain cases are clearly routed to human review.
- Important decisions are inspectable and auditable.
- Resolved cases remain visible and understandable.
- Multiple valid datasets can use the same product pipeline.
- AI enhances investigation but never becomes the uncontrolled source of truth.
- The final product is strong enough for confident Buildathon demonstration.

12. Final Direction

We will not follow the old roadmap mechanically anymore. Phases 1–11 successfully delivered the foundation. The project now enters deliberate product development.

Final sequence:

Audit → Correct Core Workflow → Refine UI/UX → Generalize Data Handling → Improve AI Experience → Add Settlement Q&A; → Evaluate → Final Polish → Deploy

This strategy protects the working source code while allowing SettleIQ to grow from a verified MVP into a stronger and more complete Buildathon-level product.